EEE 5028 LSV FINAL PROJECT REPORT : 2023 IWLS CONTEST

Jian-Heng Liu, Zhong-Yu Sui, Hung-Ling Liu

National Taiwan University Taipei 10617, Taiwan

ABSTRACT

We selected the 2023 IWLS Programming Contest [1] as the topic. The goal of the contest was to synthesize AIGs with the minimal number of And-Inverter Graphs (AIGs) and two-input XOR-AND nodes (XAIGs), for a set of completely-specified multi-output Boolean functions represented using truth tables. We used ML techniques to try to convert the output functions in truth table format into AIG format, and implemented simulated annealing with ABC [2] which is a powerful software system for synthesis and window rewriting [3] respectively to resynthesis the circuit. The final results were close to the best solutions from the competing teams.

***Index Terms*—** synthesis, resynthesis, Machine learning, abc, simulated annealing, window rewriting.

1. INTRODUCTION

In this final project, we chose the theme of the IWLS Contest 2023. The contest involves synthesizing the truth table of a complete specified function into the smallest number of AIGs and XAIGs. The challenge lies in how to synthesize an AIG circuit that matches the truth table while minimizing the number of used nodes. This article will focus on the synthesis of the AIG part. During the problem-solving process, we divided it into two parts: circuit generation and circuit optimization. The process starts with circuit generation from the truth table to create a good basic AIG, followed by continuous optimization of this basic AIG until we find the best AIG. In the following article, we will explain the algorithms we tried for both generation and optimization, and finally, we will present our experimental results and summarize our conclusions.

2. ALGORITHM

Regarding how we address this problem, our process is as shown in the fig.1 below. It starts with circuit generation from the truth table to AIG, which includes using ABC command combined with BDD reordering and machine learning. We will focus on the ideas and implementation of machine learning. Then, from AIG to the best AIG is circuit optimization. In this part, we used simulated annealing and window rewriting to optimize the circuit. Next, we will provide detailed explanations of these algorithms.

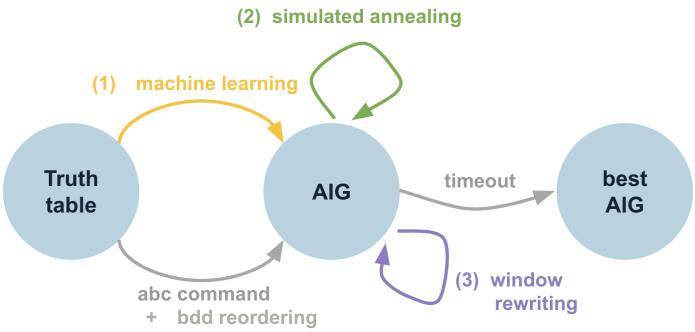


Fig. 1. algorithm

3. CIRCUIT GENERATION

In this section, we divided our approach into two types: machine learning, and techniques involving ABC command combined with BDD reordering.

3.1. ABC command with BDD reordering

Firstly, we referred to several command combinations proposed by participants in the IWLS 2022 competition [4], where ABC can generate AIGs from truth tables. From these, we selected three combinations that we believe have the most potential, as follows:

- ”collapse; sop; strash;”
- ”fx; strash;”
- ”collapse; sop; fx; strash;”

In addition to solely using ABC commands, we also wanted to explore ways to optimize circuit generation. Therefore, we looked into the BDD reordering method proposed by the UC Berkeley participants in IWLS 2022. This involves using reordering techniques to find a more efficient variable ordering while ABC is converting the truth table into BDD. Then, the BDD with the new ordering is converted into AIG. After

testing various combinations, we found that using BDD reordering in conjunction with the third type of ABC command combination yielded the best generation results. Therefore, the final process of this method is as follows:

- "collapse; bdd_reorder; sop; fx; strash;"

3.2. Circuit Neural Network

The main idea is to give a skeleton of the AIG circuit and learn the details of the circuit. Circuit neural network is a framework which is proposed by Google DeepMind in the IWLS contest [1]. It initializes levels of gates, including AND, OR, and NAND gates, with the fan-in of each gate undetermined. To learn the probability of a gate being the fan-in of another, all the processes should be differentiable. Notice the original definition of boolean gates is not differentiable, thus we view AND, OR, and NOT to be the intersection, union, and complement operations on probabilities. That is

$$AND(p,q) = p * q \tag{1}$$

$$OR(p,q) = p + q - p * q \tag{2}$$

$$NOT(p) = 1 - p \tag{3}$$

. Also, determining which fan-in is used would require argmax operations, while argmax is not differentiable. Here use Gumbel-softmax trick, which randomly chooses a fan-in as input, and the randomness is the same as the probability distribution. Gumbel-softmax trick is differentiable and, thus can be the alternative for argmax operation.

3.3. Reinforment Learning

We notice that reinforcement learning is commonly used in the resynthesis process. Thus, we want to check whether reinforcement learning can help in the generation process, too. [5] The basic idea for reinforcement learning contains actors, critics, and the environment. Actors have the observation from the environment as input and return the corresponding action. Critics would evaluate the action under the observation, and give a score on it. Actors then can use the score to update the model so that it can perform better. The environment would be initialized and updated by actions, and give an observation out.

We use PPO [6] as our actor, which is a well-developed technique proposed by OpenAI. The environment contains two parts. The goal part is the truth table of primary outputs. The node part is the truth table of primary inputs and determined nodes. Since the AIG graph can have too many gates, and make the actor consider too many choices, we would only remember some AIG nodes in the node part. The determination signal would be sent when the amount of composed AIG node has reached a given value $maxstep$, or all the primary output has been composed

A valid action includes choosing two remembered AIG nodes as fan-in, and determining whether a negative edge is used on fan-in. Action should also consider which AIG node currently in the node part should be forgotten and replaced by a newly composed AIG node.

The critic would return the reward as follows:

$$\begin{cases} 2 - \frac{numnode}{maxstep} & \dots \quad case1 \\ 0.5 * numnode^{-2} & \dots \quad case2 \\ 0 & \dots \quad otherwise \end{cases} \tag{4}$$

where $numnode$ is the amount of AIG that has been composed before this primary output.

Case 1 means the newly composed AIG node is in the goal part. However, with only the reward from case 1, it might be hard for the actor to get any score at the start of training, and thus the actor would not know how to do better. Thus, we also provide the ABC synthesized AIG as a reference, and give little reward to the actor as it composed some node in the provided AIG, which is case 2. The sum of the reward in case 2 should not exceed any possible reward given in case 1, since a smaller AIG is always better as long as it's correct. Though the provided AIG is not optimized, it still functions correctly and can lead the actor to have a correct AIG first.

4. CIRCUIT OPTIMIZATION

In the optimization part, we tried simulated annealing and window rewriting algorithm. Below is the detailed description.

4.1. Simulated Annealing

Algorithm 1 Simulated Annealing

Input: Original Circuits C
Output: Optimized Circuits C_{opt}

```
T ← T_max
E ← E(C) the number of node of the initial circuit
while (T > T_min) do
    C_new ← generate a resynthesized candidate circuit
    E_new ← compute the number of node of new circuit
    ΔE ← E_new − E
    if random() < Probability(ΔE, T) then
        C ← C_new
        E ← E_new
    end if
    T ← T/α cooling the temperature
end while
return C
```

Simulated annealing is a probabilistic technique used for finding an approximate solution to an optimization problem. It starts with a random solution and then "heats up" the system to allow for more random exploration. During the process,

the algorithm randomly explores solutions. If a new solution is better than the current one, it's accepted. If it's worse, it might still be accepted with a probability that decreases as the "temperature" drops. Algorithm 1 is the algorithm for it

This algorithm introduces a stochastic element, meaning the algorithm doesn't just move towards better solutions, preventing it from getting stuck in local optima. The key reason simulated annealing is effective in finding optimal solutions is its ability to escape local optima. Standard hill-climbing algorithms can get stuck in these local optima. Simulated annealing's probabilistic acceptance of worse solutions allows it to jump out of these local optima and explore more of the solution space, as shown in Fig. 2 [7].

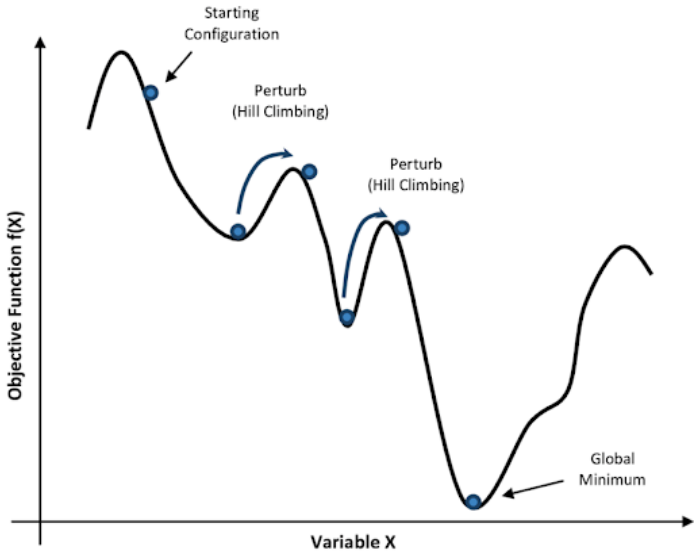


Fig. 2. Simulated Annealing

In our use of simulated annealing, there are several noteworthy aspects. First, since most resynthesis algorithm does not produce worse solutions, often resulting in no change in the number of nodes, we consider all cases where the optimized node count is less than a certain value as having no optimization. We need to calculate probabilities to decide whether to accept such solutions. Furthermore, the most crucial part is how to resynthesize the old circuit and select candidate circuits. The resynthesis algorithm used significantly affects our ability to find better solutions. Therefore, we ultimately decided to use several resynthesis commands that we found feasible and powerful in ABC, including the following:

- common resynthesis : rewrite, refactor, resub
- deepsyn
- transduction, dc2
- orchestrate

The common resynthesis part consists of a combination of classic rewrite, refactor, and resubstitution, representing a more traditional and classical approach. DeepSyn is a novel and powerful resynthesis command in ABC [2], composed of several highly effective resynthesis combinations, and it's the best solution we have. Transduction is a method for redundant node removal using don't cares, proposed by the UC Berkeley team in IWLS 2022 [4], and it also performs very well. Finally, there's Orchestrate [8], a new method proposed in 2023 by Alan Mishchenko's laboratory, which is also a potential option for our resynthesis.

4.2. Window rewriting

Previous AIG rewriting methods face challenges with exponentially growing computational complexity and database requirements. Our approach builds on the resubstitution algorithm covered in lecture07 p.42, leveraging don't cares to optimize sub-circuits using simulation to rule out some subset. We implemented the method proposed in [3], which optimizes AIGs through localized window rewriting.

4.2.1. Boolean Rewriting

The process is as illustrated in Fig. 3

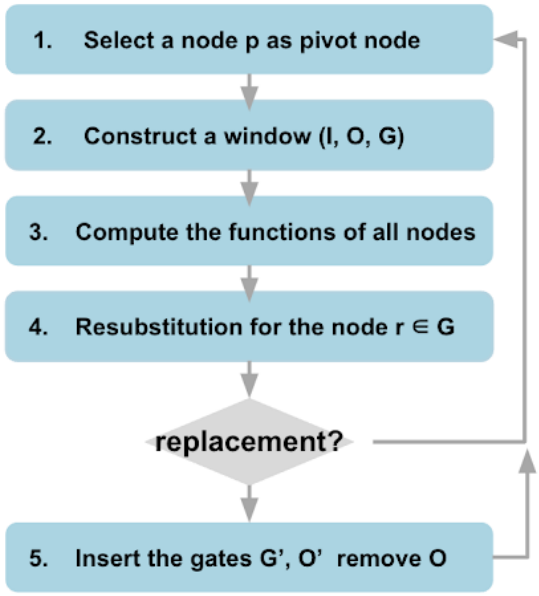


Fig. 3. Boolean Rewriting

The conceptual steps can be implemented with different strategies. The details on how we build the Windows in Step 2 and Step3 Compute the functional output, resynthesis in Step 4 are covered in the next section.

According to the following steps after given a node p, a integer l for limited distance from p, it would return (I,O,G) to represent input, output, inner node in the window:

1. Identify and collect reconvergence.

   Use BFS to identify a node m , called meet, reachable from two fanins of p in at most l steps and add all nodes on the two paths from p to m to G.

2. Collect inputs.

   Iterate over all fanins of the nodes in G and add all nodes without marks to I . If $|I| > 6$, terminate without returning a result.

3. Expand towards TFI.

   As long as $|I| \leq 6$, repeat two steps:

   (a) first perform all cost-free expansions of I;

   (b) then choose a node g from I such that $|EXPAND(I, g) \leq 6$ and g has a highest fanout count within G. Expand I with g, and go to (a).

   If no further expansion is possible, go to the next step.

4. Expand towards TFO.

   Sort all nodes in $I \cup G$ into vector v by their distance from I from lowest to highest. For each node, if a fanout g is an inner node and not marked, but all its fanins are marked, then add g to G, mark it, and sort it into vector v.

5. Identify and collect window outputs.

   Define a reference counter for each node in the window. Iterate over all nodes in G and increment the counters of all fanins of $g \in G$. Observe their reference counters, and add a node to output O if its reference counter is lower than its fanout count so that all fanout of inner node $g \in G$ are in the window.

6. Topologically sort.

   Sort the nodes in G in topological order and return the triple (I, O, G)

4.2.2. Don't care computing

As mentioned in the lecture algorithm, we represent functions as truth tables. In a window, pick a node r and first simulate to get the original output functions, then complement the function of r, re-simulate the TFO(r) to find the ODC. The ODC can be computed by:

$$d_{c_r} = \neg \bigvee_{o_i \in O} f_{o_i} \oplus f'_{o_i} \tag{5}$$

Therefore, that $f_{on} = f_r \wedge \neg dc_r$ and $f_{off} = \neg f_r \wedge \neg dc_r$, which is necessary for resynthesis.

4.2.3. Heuristic Boolean Resynthesis

The algorithm is based on a recursive decomposition that classifies divisors on their intersections with the on-set and the off-set of a target function.

To classify divisors, we categorize divisors into three types: positive unate, negative unate, and binate (when a divisor is neither positive unate nor negative unate). Note that the unateness property is independent for d and ¬d, a literal is either a divisor or a negated divisor. If combine binate divisors by AND gate, we could create some positive/negative unate divisor pairs.

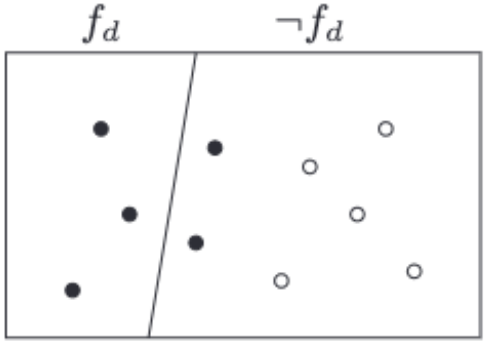


Fig. 4. d is positive unate

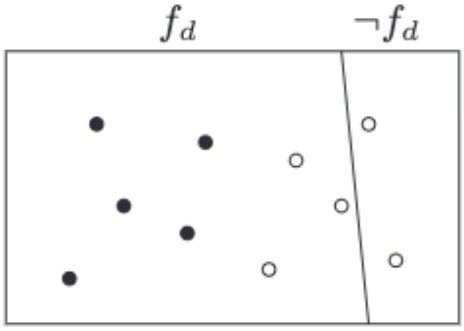


Fig. 5. ¬d is negative unate

A positive unate divisor d (or its negation ¬d) satisfies(Fig.4):

$$f_d \wedge f_{off} = 0 \tag{6}$$

A negative unate divisor d (or its negation ¬d) satisfies(Fig.5):

$$f_d \wedge f_{on} = 0 \tag{7}$$

After classifying divisors, we select a target node r to be being substituted based. For example, if we use 2 positive unate divisors(or divisor pairs) d1, d2 and $\neg(f_{d1} \vee f_{d2}) \wedge f_{on} = 0$, then we find a replacements to be OR-type 1-resub.

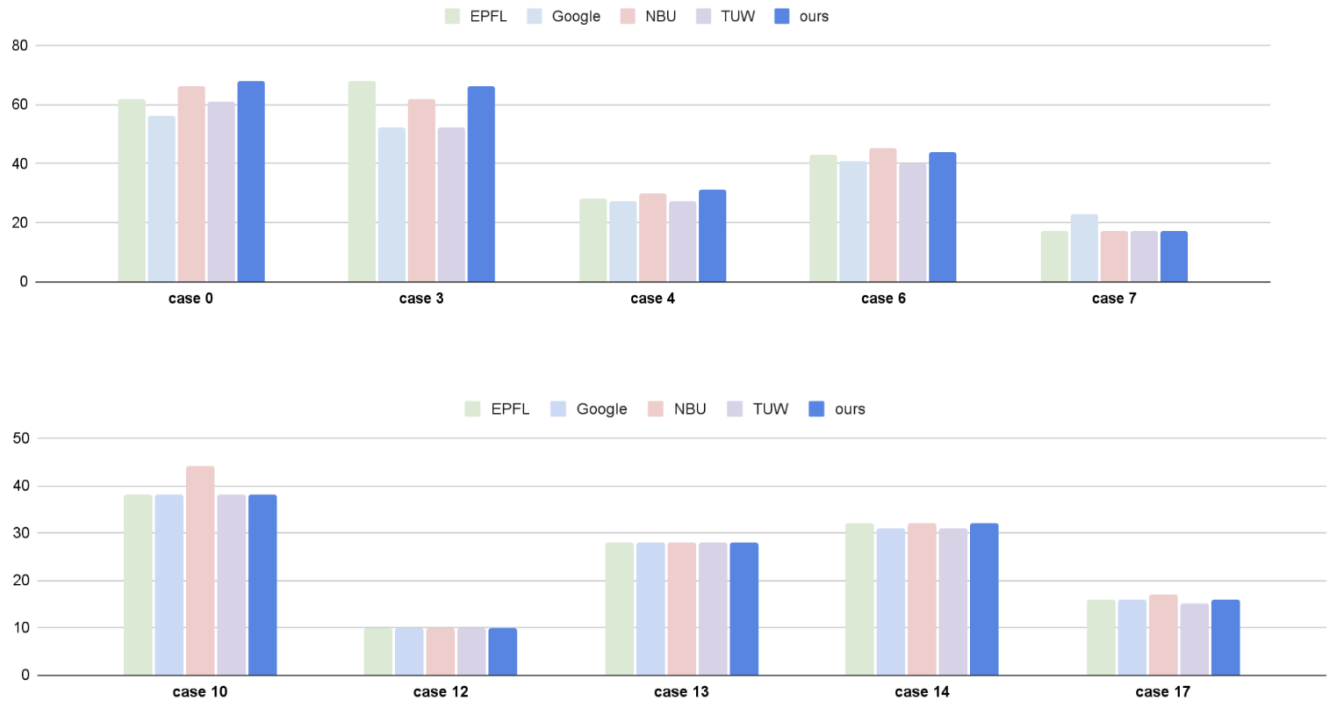


Fig. 6. Simulated annealing result for each case

There is AND-type 1-resub if $(\neg f_{d1}) \wedge (\neg f_{d2}) \wedge \neg f_{off} = 0$. Update circuit if improves. This iteratively attempts to simplify circuitry around different target nodes.

5. EXPERIMENTAL RESULTS

For the Experimental Results section, we divided it into three parts: machine learning, simulated annealing, and window rewriting, to present the performance outcomes of each.

5.1. Machine learning

5.2. Simulated annealing

We randomly selected 10 small test cases for experimentation because larger test cases required too much time. The experiment started with generating AIGs using ABC commands combined with BDD reordering, followed by simulated annealing on these AIGs. The process involved testing various initial parameters for simulated annealing and ultimately selecting the best results. The experimental outcomes for each case are shown in Fig. 6. It is evident that our simulated annealing is very competitive for each case, closely matching the performance of other IWLS 2023 participants, and in some cases, even producing the best solutions.

The sum of all test results are presented in Fig. 7. Although slightly behind other teams, our scores are very close, and considering the shorter actual execution time compared to other competitors, the results are quite satisfactory.



Fig. 7. Sum of all simulated annealing case

5.3. Window rewriting

The image in Fig.8 shows the experimental results of our window rewriting. We randomly sampled 10 data points and

10	before : 331 after : 331	15	before : 6194 after : 6189 (-5)
11	before : 3257 after : 3257	16	before : 3029 after : 3029
12	before : 3257 after : 3257	17	before : 33 after : 33
13	before : 142 after : 142	18	before : 19348 after : 19318 (-30)
14	before : 86 after : 86	19	before : 212 after : 205 (-7)

Fig. 8. window rewriting result

found the optimization results are not as good as expected. We believe there are three ways this could be enhanced further:

- Try resubstitution with additional divisors in more complex combinations
- Even when the cost is unchanged, still update the circuit to avoid getting stuck in local minima
- Apply more resubstitution passes on the same window to find incremental improvements

6. KEY RESEARCH CONTRIBUTIONS

name	***works***	***percentage***
Jian-Heng	Machine learning	33.3%
Zhong-Yu	Simulated annealing	33.3%
Hung-Ling	Window rewriting	33.3%

Basically, everyone has participated in each work. Only the people who have a higher participation rate are listed.

7. CONTACT INFORMATION

- Jian-Heng: b08901032@ntu.edu.tw
- Zhong-Yu : johnsonsui3@gmail.com
- Hung-Ling: r12921045@ntu.edu.tw

8. REFERENCES

[1] "2023 iwls programming contest," 2023. [Online]. Available: https://www.iwls.org/iwls2023

[2] "Abc : A system for sequential synthesis and verification." [Online]. Available: https://people.eecs.berkeley.edu/ alanmi/abc/

[3] H. Riener, S.-Y. Lee, A. Mishchenko, and G. De Micheli, "Boolean rewriting strikes back: Reconvergence-driven windowing meets resynthesis," in *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2022, pp. 395–402.

[4] "2022 iwls programming contest," 2022. [Online]. Available: https://www.iwls.org/iwls2022

[5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. The MIT Press Cambridge, Massachusetts London, England, 2018.

[6] "Proximal policy optimization." [Online]. Available: https://openai.com/research/openai-baselines-ppo

[7] O. Ghasemalizadeh, M. Khaleghian, and S. Taheri, "A review of optimization techniques in artificial networks," *International Journal of Advanced Research*, vol. 4, pp. 1668–1686, 09 2016.

[8] Y. Li, M. Liu, M. Ren, A. Mishchenko, and C. Yu, "Dag-aware synthesis orchestration," *arXiv preprint arXiv:2310.07846*, 2023.